Verilog coding
Day-02

Design and simulate a **Half adder, Full Adder, Half Subtractor, Full Subtractor** using Verilog HDL.

- Implement using:
  - Gate-Level Modeling
  - Dataflow Modeling
  - Behavioral Modeling
- Write a testbench to verify correctness.

1. HALF ADDER

Dataflow HA

```
module half_adder_df(input a, b, output sum, carry);
  assign sum   = a ^ b;
  assign carry = a & b;
endmodule
```

Gate-Level HA

```
module half_adder_gl(input a, b, output sum, carry);
  xor (sum, a, b);
  and (carry, a, b);
endmodule
```

Behavioral HA

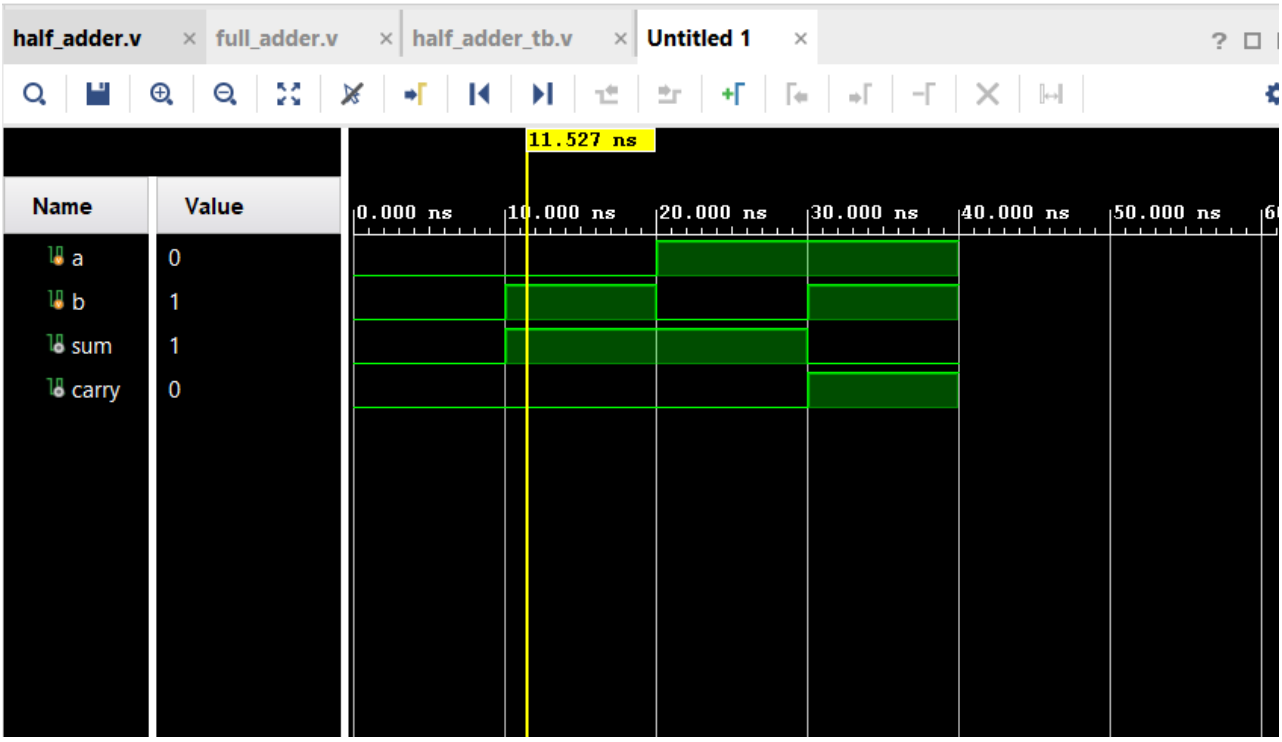
```
module half_adder_bh(input a, b, output reg sum, carry);
  always @(*) begin
    sum   = a ^ b;
    carry = a & b;
  end
endmodule
```

TestBench HA

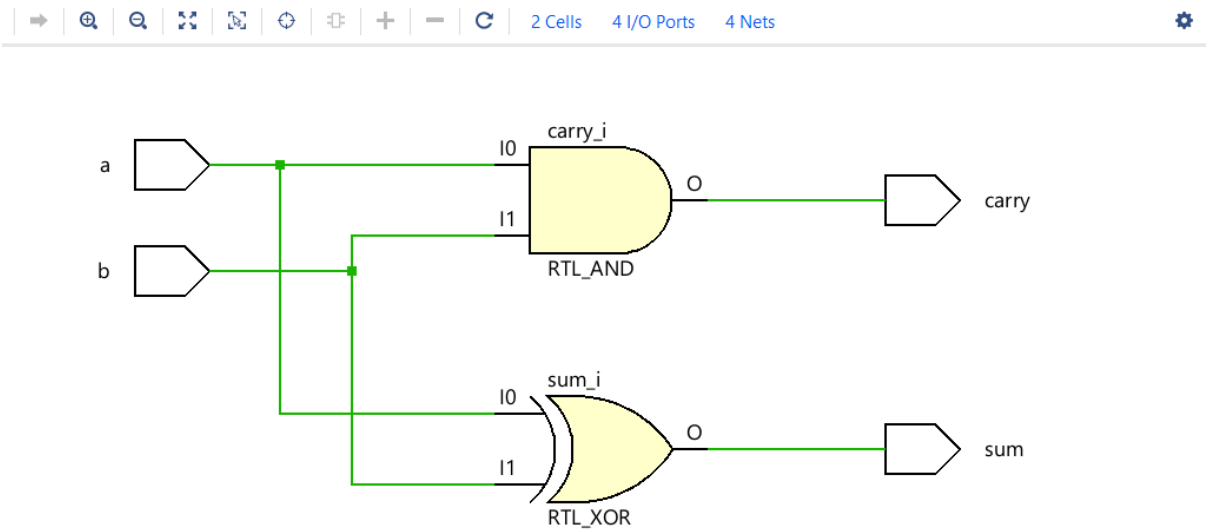
```
module half_adder_tb();

  reg a, b;
  wire sum, carry;

  half_adder_gl uut (
    .a(a),
    .b(b),
    .sum(sum),
    .carry(carry)
  );
```

```
 initial begin
  $monitor("%b %b |  %b    %b", a, b, sum, carry);

  a=0; b=0; #10;
  a=0; b=1; #10;
  a=1; b=0; #10;
  a=1; b=1; #10;

  $finish;
 end

endmodule
```

waveform HA



RTL DESIGN HS



2. FULL ADDER

Dataflow FA

```
module full_adder_df(input a, b, cin, output sum, cout);
  assign sum  = a ^ b ^ cin;
  assign cout = (a & b) | (b & cin) | (a & cin);
endmodule
```

Gate-Level FA

```
module full_adder_gl(input a, b, cin, output sum, cout);
  wire w1, w2, w3;

  xor (w1, a, b);
  xor (sum, w1, cin);

  and (w2, a, b);
  and (w3, w1, cin);
  or  (cout, w2, w3);
endmodule
```

Behavioral FA

```
module full_adder_bh(input a, b, cin, output reg sum, cout);
  always @(*) begin
    sum  = a ^ b ^ cin;
    cout = (a & b) | (b & cin) | (a & cin);
  end
endmodule
```

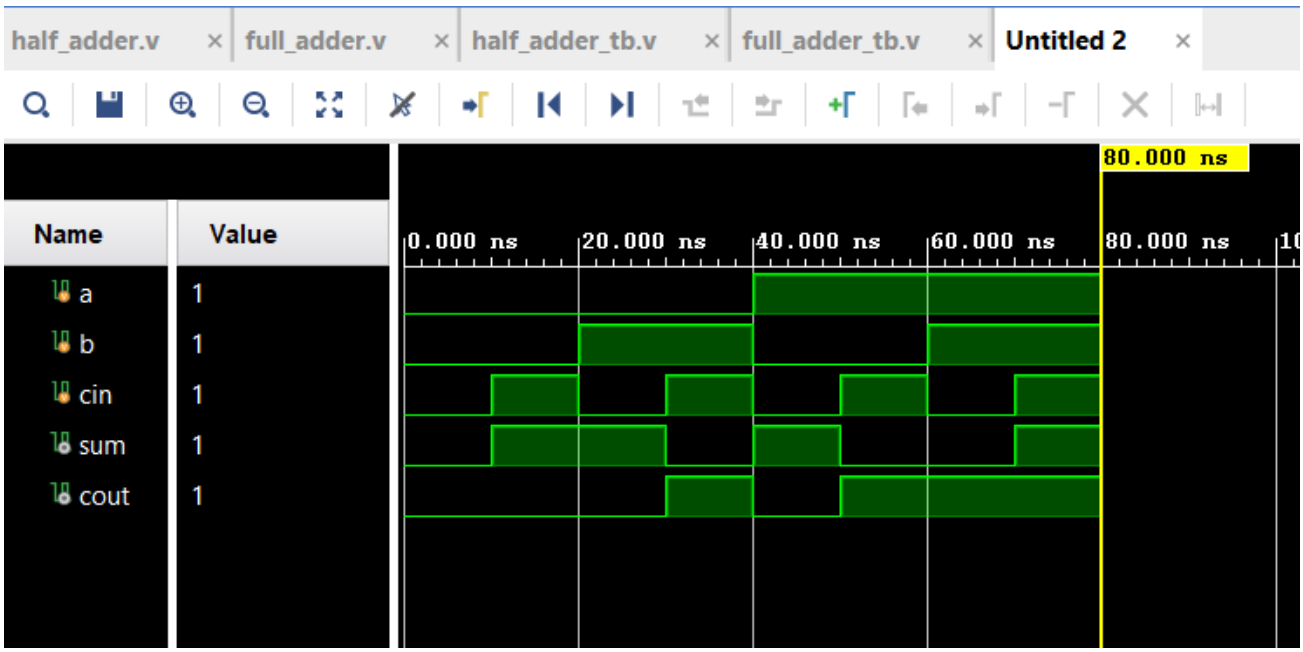
Structural (Using Component Instantiation) FA

```
module full_adder_struct(
   input a, b, cin,
   output sum, cout
);
  wire s1, c1, c2;
half_adder_gl HA1 (a, b, s1, c1);
half_adder_gl HA2 (s1, cin, sum, c2);
assign cout = c1 | c2;
endmodule
```

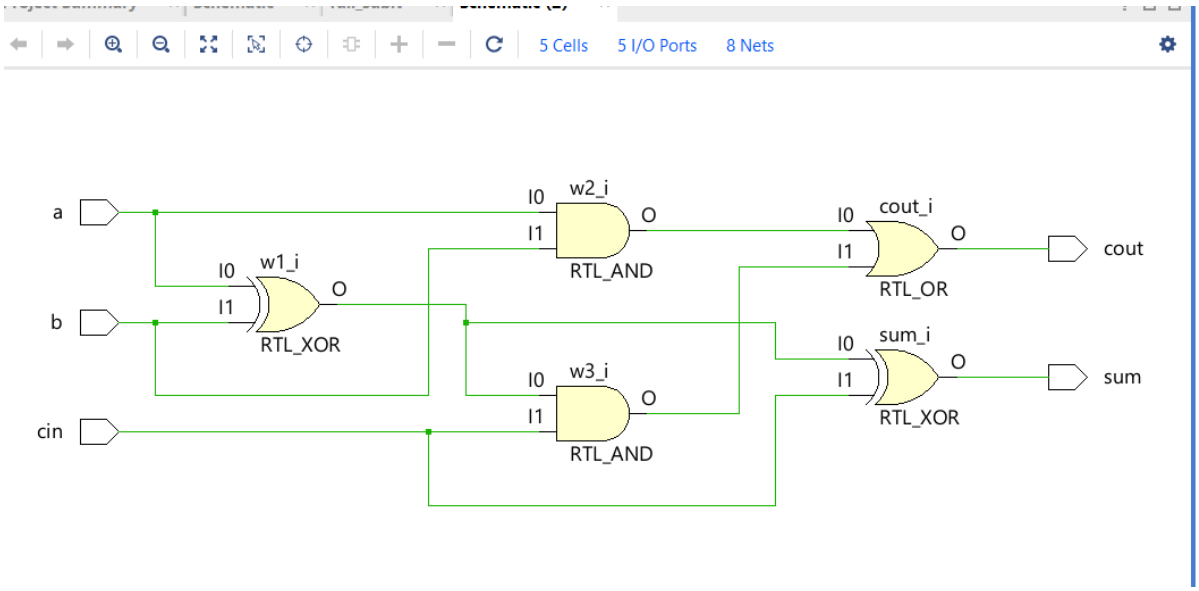
TestBench FA

```
module full_adder_tb();
  reg a, b, cin;
  wire sum, cout;
  full_adder_gl uut (
    .a(a),
    .b(b),
    .cin(cin),
    .sum(sum),
    .cout(cout)
  );
initial begin
     $monitor("%b %b  %b  |  %b   %b", a, b, cin, sum, cout);

    a=0; b=0; cin=0; #10;
    a=0; b=0; cin=1; #10;
    a=0; b=1; cin=0; #10;
    a=0; b=1; cin=1; #10;
    a=1; b=0; cin=0; #10;
    a=1; b=0; cin=1; #10;
    a=1; b=1; cin=0; #10;
    a=1; b=1; cin=1; #10;
    $finish;
  end
endmodule
```

WaveForm FA



RTL design FA



---

3. HALF SUBTRACTOR

Dataflow HS

```
module half_sub_df(input a, b, output diff, borrow);
  assign diff   = a ^ b;
  assign borrow = (~a) & b;
endmodule
```

Gate-Level HS

```
module half_sub_gl(input a, b, output diff, borrow);
  xor (diff, a, b);
  not (na, a);
```

```
  and (borrow, na, b);
endmodule
```

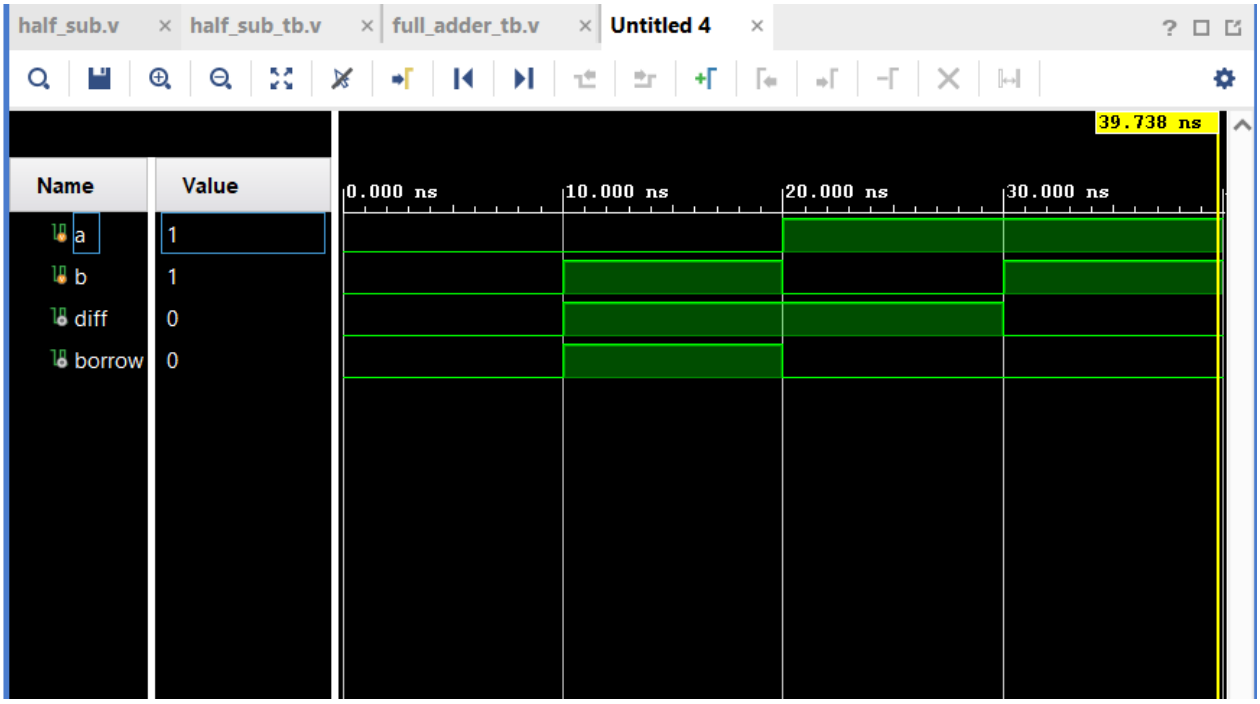
Behavioral HS

```
module half_sub_bh(input a, b, output reg diff, borrow);
  always @(*) begin
    diff   = a ^ b;
    borrow = (~a) & b;
  end
endmodule
```

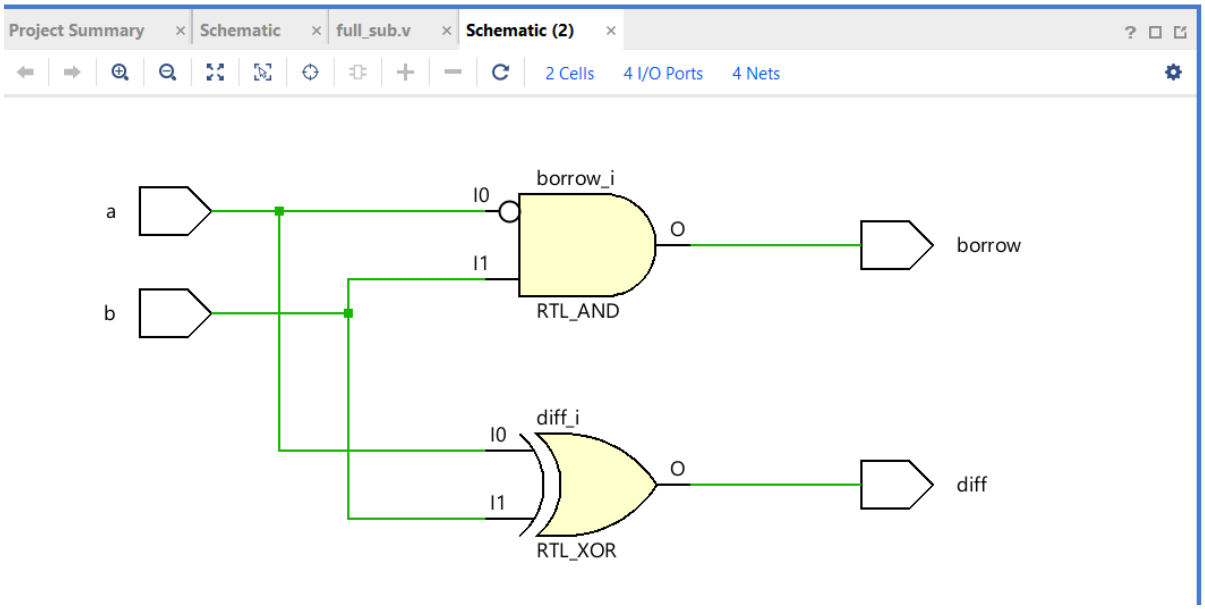
TestBench HS

```
module half_sub_tb;
  reg a, b;
  wire diff, borrow;
  half_sub_df uut (
    .a(a),
    .b(b),
    .diff(diff),
    .borrow(borrow)
  );
  initial begin
    $monitor("%b %b |  %b     %b", a, b, diff, borrow);
    a=0; b=0; #10;
    a=0; b=1; #10;
    a=1; b=0; #10;
    a=1; b=1; #10;
    $finish;
  end
endmodule
```

WaveForm HS



RTL design HS



---

4. FULL SUBTRACTOR

Dataflow FS

```
module full_sub_df(input a, b, bin, output diff, borrow);
  assign diff   = a ^ b ^ bin;
  assign borrow = (~a & b) | (~(a ^ b) & bin);
endmodule
```

Gate-Level FS

```
module full_sub_gl(input a, b, bin, output diff, borrow);
  wire w1, w2, w3, na;

  xor (w1, a, b);
  xor (diff, w1, bin);

  not (na, a);
  and (w2, na, b);
  not (w3, w1);
  and (w3, w3, bin);

  or (borrow, w2, w3);
endmodule
```

Behavioral FS

```
module full_sub_bh(input a, b, bin, output reg diff, borrow);
  always @(*) begin
    diff   = a ^ b ^ bin;
    borrow = (~a & b) | (~(a ^ b) & bin);
  end
endmodule
```

Structural (Using Component Instantiation) FS

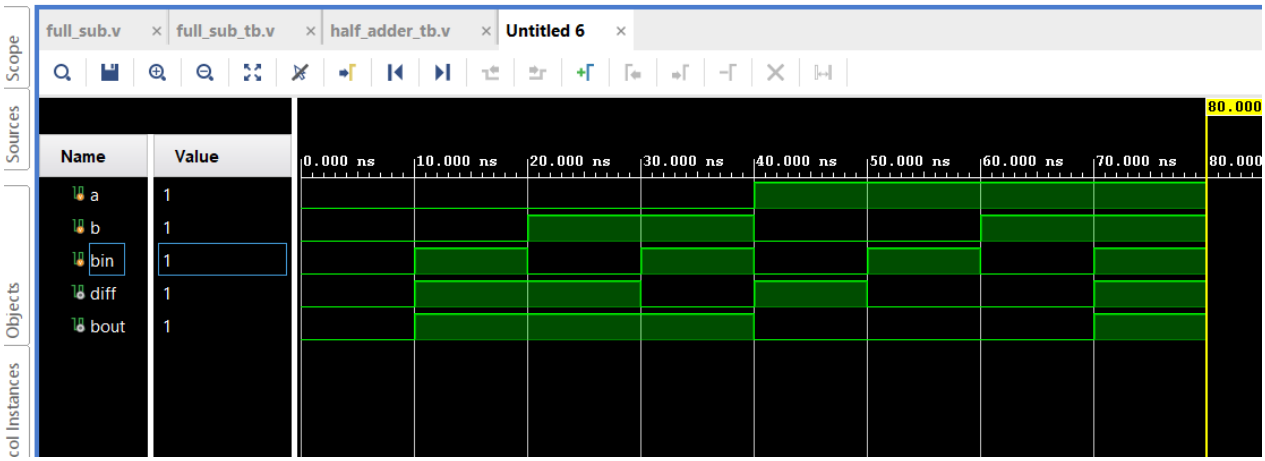
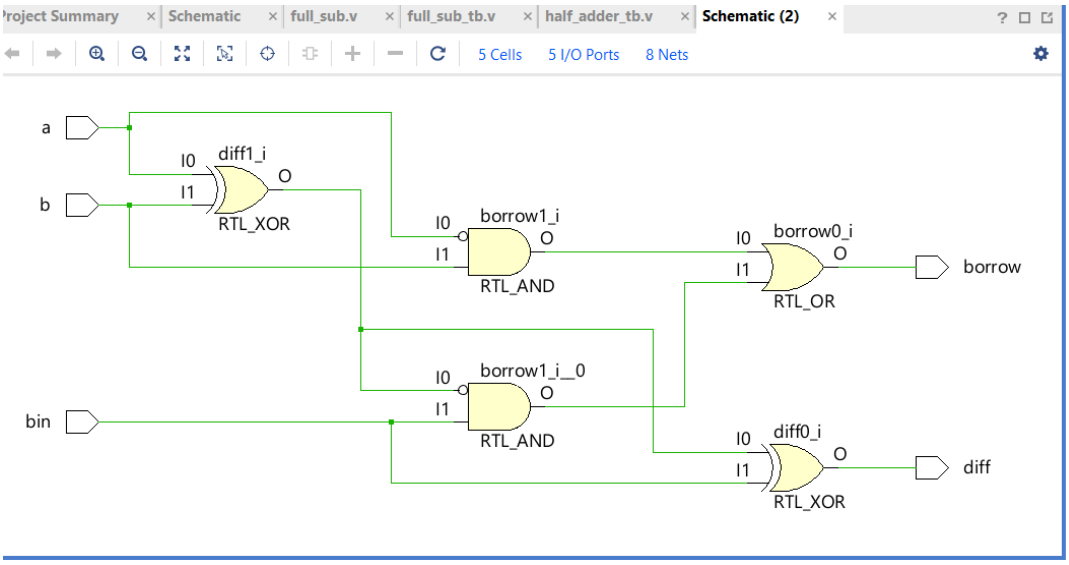
```
module full_subtractor_struct(
  input a, b, bin,
  output diff, bout
);
  wire d1, b1, b2;
  half_sub_df hs1 (
    .a(a),
    .b(b),
    .diff(d1),
    .borrow(b1)
  );
  half_sub_df hs2 (.a(d1), .b(bin), .diff(diff), .borrow(b2) );
  assign bout = b1 | b2;
endmodule
```

Test bench FS

```
module full_subtractor_struct(
  input a, b, bin,
  output diff, bout
);
```

```
  wire d1, b1, b2;
  half_sub_df hs1 (
    .a(a),
    .b(b),
    .diff(d1),
    .borrow(b1)
  );
  half_sub_df hs2 (.a(d1), .b(bin), .diff(diff), .borrow(b2) );
  assign bout = b1 | b2;
endmodule
```

waveform FS



RTL design FS